

METODOLOGIA DE ENGENHARIA · HUMANO + IA

Maestro

Handbook do Modelo Operacional

Fundamentos por elemento — 12 capítulos, 5 camadas.



PLATAFORMA GH DARU TECNOLOGIA
DOCS/HANDBOOK · GERADO DA FONTE VERSIONADA
1 HUMANO REGENDO N AGENTES DE IA

Handbook do Modelo Operacional — Fundamentos por elemento

Manual de referência dos elementos do modelo operacional (docs/governance/modelo-operacional.md). Um capítulo por elemento, cada um com a mesma anatomia: fundamentação teórica → frameworks avaliados → recomendação de uso no nosso contexto (1 humano orquestrando N agentes de IA).

Este handbook é **didático e fundamentado**; complementa — não substitui — os outros três documentos:

DOCUMENTO	PAPEL
governance/modelo-operacional.md	Normativo — a regra vigente (o quê é obrigatório)
handbook/ (aqui)	Fundamentos — por que a regra é essa; teoria + frameworks + recomendação
research/jornada-aprendizado-modelo-operacional.md	Diário — os insights construídos ao criticar cada elemento
research/resultado-pesquisa-*-avaliacao.md	Pesquisa — a síntese citada que embasa tudo

Anatomia obrigatória de cada capítulo (7 seções)

- Pergunta central** — a pergunta que o capítulo responde.
- Fundamentação teórica** — o conceito, sua origem e o princípio que o sustenta.
- Frameworks / abordagens avaliados** — o que existe lá fora, comparado, com veredito.
- Recomendação de utilização** — como aplicamos no contexto 1 humano + N agentes; ligação com o modelo-operacional.md e com a Constituição.
- Conexões** — como o elemento se liga aos demais capítulos.
- Insight da jornada e impacto no modelo** — o que refinamos ao avaliar criticamente e o efeito normativo (ADR/versão), com link ao diário.
- Fontes** — citadas, com data.

Índice (ordem de leitura = ordem de dependência)

CAP.	ELEM.	TÍTULO	RESUMO (IDEIA CENTRAL)
01	[1]	Princípio operacional central	IA escreve · humano decide/aprova · verificação independente valida. O que torna o irreversível seguro de delegar é a reversibilidade engenheirada .
02	[10]	Evidência: DORA/SPACE	Velocidade e estabilidade não são trade-off . Alavanca: lote pequeno + reversibilidade . Bússola, não painel.

CAP.	ELEM.	TÍTULO	RESUMO (IDEIA CENTRAL)
03	[2]	Spec-Driven	A spec é a fonte de verdade (input que gera código, não descrição). Raias leve/plena/infra por ambiguidade × raio × irreversibilidade .
04	[3]	Fluxo agentic e economia de contexto	explore→plan→code→commit + subagentes + /clear + revisor fresco = economia de contexto . A spec é o contexto integrador.
05	[4]	Orquestração de agentes	Map-reduce cognitivo : reduce do orquestrador; corte nas costuras (bounded contexts). Espectro workflow↔agent; menor autonomia que resolve .
06	[5]	Papéis e RACI	Papéis como modos. Delega-se R/C/I ; nunca o A . Humano Accountable pela política/gates/critérios — não por item .
07	[6]	Cerimônias e cadência	Cerimônia = função , não reunião. Retro amplificada por agentes. WIP = atenção humana . Shape Up + Kanban.
08	[7]	Entregáveis e artefatos	Artefato vive se é input consumido com forcing function (ou imutável, como o ADR). Não duplicar função já servida.
09	[8]	Definition of Ready / Done	"Done" precisa ser verificável autonomamente ; converter juízo em check . Verde ≠ certo — global fica com o humano.
10	[9]	Gates e classes de risco	Gate proporcional a irreversibilidade × impacto . Uniforme = funil ou catástrofe. Reversibilidade rebaixa a classe .
11	[11]	Rastreabilidade	spec ↔ PR ↔ teste ↔ journey = memória durável (sobrevive ao reset do agente). Emerge do workflow, sem ferramenta.
12	[12]	Governança leve	Aprende sem inchar : núcleo firme + periferia evoluível + YAGNI . A jornada <i>foi</i> o loop de governança em ação.

Nota: o número do capítulo (ordem de leitura) ≠ o rótulo do elemento [x] (nossa linguagem compartilhada na jornada). A tabela mapeia os dois.

Capítulo 01 — Princípio operacional central (elemento [1])

IA para explorar, propor e escrever; humano para especificar, decidir e aprovar; testes, gates e revisão independente para validar.

1. Pergunta central

Se um agente é capaz de tomar qualquer decisão — inclusive registrar alternativas e racional impecáveis — o que ainda exige o humano?

2. Fundamentação teórica

Quando é o agente que escreve o código, o eixo de controle se desloca: o que garante que o software faz o que o negócio quer não é mais digitar código — é a clareza da **intenção** e a força da **verificação**. Três pilares:

- 1. **A intenção vive na especificação**, não no código nem no prompt (base do [2]). O humano dirige e refina; o agente escreve.
- 2. **Quem executa não é quem verifica**. A verificação passa por um revisor independente, de preferência em *contexto fresco* — segregação de funções aplicada a agentes.
- 3. **Prove, não declare**. "Pronto" exige evidência que o agente gere e um gate confira.

Accountability x capacidade. Capacidade de *raciocinar* não é o mesmo que *responder pelas consequências*. O gate humano não mede a qualidade do raciocínio do agente; ele **localiza a responsabilidade**.

Ex-ante x ex-post. Um registro auditável é *ex-post* — descreve a decisão depois de tomada. Um gate é *ex-ante* — barra antes de executar. Para uma ação **irreversível**, auditar não impede o dano.

A alavanca real: reversibilidade. O que torna uma ação irreversível *segura de delegar* não é a aprovação — é convertê-la em reversível (backup, dry-run, staging, soft-delete). O gate humano sempre foi um **proxy** de "torne reversível ou olhe antes".

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Human-in-the-loop / mixed-initiative (Horvitz)	Humano no ponto de decisão para ações consequentes	Adotado — espinha do princípio
Política declarativa allow/deny/ask (Const. §IV; OpenAI Agents; OPA)	Decidir classes de ação autônoma, não cada instância	Adotado — humano decide a política, que fica registrada

ABORDAGEM	O QUE OFERECE	VEREDITO
ADR (Nygard)	Registro auditável: decisão + contexto + consequências + alternativas	Adotado — protocolo de registro de decisão
Reversibility patterns (backup/dry-run/staging/soft-delete; git checkpoint/rewind)	Tornar o irreversível reversível	Adotado — requisito de DoD para ação irreversível
Taxonomia de classes de risco (Const. §IV)	Grada o gate por risco	Adotado — mapa dos gates humanos

4. Recomendação de utilização (1 humano + N agentes)

- **O humano decide a política de delegação, e ela vai ao registro** — a responsabilidade sobe da instância para a política (`allow/deny/ask`).
- **Verificação independente valida** — revisor em contexto fresco (`[3]`).
- **Ação irreversível exige reversibilidade engenheirada** antes de delegar (DoD de infra — `modelo-operacional.md` §7, §8).
- Gates indelegáveis: aprovar spec, plan, merge, autorizar deploy/migração.

5. Conexões

- `[2]` **Spec-Driven** — a spec é onde a intenção humana vive.
- `[3]` **Fluxo agentic** — reversibilidade = o checkpoint/rewind/git; verificação = revisor fresco.
- `[10]` **DORA** — "recuperável > cauteloso" é a evidência empírica da reversibilidade.
- `[9]` **Gates/risco** — a taxonomia que operacionaliza o gate.

6. Insight da jornada e impacto no modelo

Refinado ao ser **criticado**: de "toda decisão pode ser automatizada com registro" → "o humano delega e a delegação vai ao registro" → **reversibilidade é o que torna o irreversível seguro de delegar**. Sem impacto normativo direto (o princípio já constava); alimentou os gates de reversibilidade formalizados no `[2]` /§7. Diário: `[1]` em `research/jornada-aprendizado-modelo-operacional.md`.

7. Fontes

- Anthropic — *Building effective agents*: <https://www.anthropic.com/engineering/building-effective-agents>
- Claude Code — *Best practices*: <https://code.claude.com/docs/en/best-practices>
- E. Horvitz — *Principles of Mixed-Initiative UI*: <https://www.microsoft.com/en-us/research/publication/principles-mixed-initiative-user-interfaces/>
- M. Nygard — *Documenting Architecture Decisions*: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- OWASP — *LLM01 Prompt Injection*: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

- Constituição da Plataforma (Princípio IV).

Capítulo 02 — Evidência: DORA / SPACE (elemento [10])

A base empírica embaixo do [1] : **velocidade e estabilidade não são um trade-off.**

1. Pergunta central

A intuição comum diz "quanto mais rápido eu entrego, mais eu quebro". **Velocidade e estabilidade são mesmo opostas — ou andam juntas?**

2. Fundamentação teórica

O programa DORA (DevOps Research and Assessment) mediu milhares de equipes e definiu **quatro métricas**, em dois pares:

- **Throughput**: frequência de deploy · lead time (commit→produção)
- **Estabilidade**: change fail rate · tempo de recuperação (rollback) — hoje às vezes com uma 5ª, *rework rate*.

Achado central, contraintuitivo: os **mesmos** times de elite são rápidos **e** estáveis, simultaneamente. "O trade-off real, no longo prazo, é entre *software melhor mais rápido e software pior mais devagar*." As quatro métricas são **outcomes** dirigidos por **capacidades**: CI, trunk-based development, testes automatizados, revisão de código, deploys pequenos.

A alavanca única: tamanho de lote pequeno. Mudança pequena → deploys mais vezes (frequência ↑), lead time curto, fácil de testar/revisar (falha ↓) e, quando falha, fácil de reverter (recuperação ↑). Padronização + automação é o que torna o lote pequeno barato; **reversibilidade** ([1]) é o que torna a falha barata. Por isso as quatro se movem juntas em vez de brigar.

3. Frameworks / abordagens avaliados

FRAMEWORK	FOCO	VEREDITO PARA SOLO+IA
DORA (4 keys + capabilities)	Desempenho de entrega + capacidades causais	Adotado como bússola qualitativa — lead time e change fail rate são os dois que mais importam para um solo
SPACE	Produtividade multidimensional (Satisfação/Performance/Atividade/Comunicação/Eficiência)	Referência — lembra que "não há uma métrica única de produtividade"; pouca prescrição

FRAMEWORK	FOCO	VEREDITO PARA SOLO+IA
DX Core 4	4 métricas prescritivas (Speed/Effectiveness/Quality/Impact)	Observado — cuidado com métricas gaméaveis (ex.: "PRs por dev")
Painel instrumentado de métricas	Dashboards de DORA	YAGNI agora — sem a quem comparar; instrumentar é débito prematuro

4. Recomendação de utilização (1 humano + N agentes)

- Usar DORA como **bússola de saúde do fluxo**, não como painel de RH ("consigo levar uma spec a produção rápido e sem quebrar?").
- Focar **lead time** e **change fail rate**; recuperação vem da reversibilidade ([1]).
- **Não instrumentar métricas** ainda (YAGNI). As capacidades (CI verde, trunk-based, testes, diffs pequenos, revisão independente) já são princípio da Constituição (V) e da DoD — a evidência só confirma que são as certas.

5. Conexões

- [1] — "recuperável > cauteloso" é a leitura empírica da reversibilidade.
- [3] — rollback rápido = o checkpoint/rewind do fluxo agentic.
- [8] **DoR/DoD** — as capacidades DORA são exatamente os itens da DoD.

6. Insight da jornada e impacto no modelo

Analogia do aprendiz: **linha de produção cognitiva** — como a fábrica pré-Ford era lenta até padronizar componentes, aqui a padronização de processo/entregáveis reduz fricção e variação. Complemento: a linha do Ford *não tinha undo barato*; a cognitiva tem — **lote pequeno + reversibilidade = rápido e estável**. Sem impacto normativo (confirma o modelo); justifica por que a DoD e as raias não são burocracia. Diário: [10].

7. Fontes

- DORA — *DORA metrics (four keys)*: <https://dora.dev/guides/dora-metrics-four-keys/>
- Swarmia — *Comparing DORA, SPACE and DX Core 4*: <https://www.swarmia.com/blog/comparing-developer-productivity-frameworks/>

Capítulo 03 — Desenvolvimento dirigido por especificação (elemento [2])

A keystone da mecânica: a spec é a fonte de verdade, não o código nem o prompt.

1. Pergunta central

No mundo tradicional, a fonte de verdade é o código — a spec e a doc *apodrecem*. O Spec-Driven inverte isso. **O que muda, quando é o agente que escreve, que faz a spec parar de apodrecer e passar a ser mais confiável que o código?**

2. Fundamentação teórica

Inversão de dependência. Em desenvolvimento tradicional a doc *descreve* o código (está a jusante → apodrece). No Spec-Driven a spec é o **input que gera** o código: se você quer código novo, precisa mudar a spec primeiro. A obsolescência deixa de ser opcional e passa a quebrar a geração — a spec vira "verdade executável".

Fluxo: `specify → clarify → plan → tasks → implement`. A spec descreve **o quê e por quê** (valor, critérios de aceite testáveis); o plan descreve **como**. Prompt vago "força o modelo a adivinhar milhares de requisitos não ditos".

Nem toda mudança merece uma spec. O valor de uma spec escala com **ambiguidade × raio de impacto × irreversibilidade**. Quando os três são baixos, a spec é cerimônia de papel (YAGNI). Daí as **raias** (`modelo-operacional.md` §3).

3. Frameworks / abordagens avaliados

FERRAMENTA	MODELO	VEREDITO
GitHub Spec Kit	1 spec/feature; fases com gate; cross-feature forte	Adotado — base do fluxo; casa com constituição + specs numeradas
OpenSpec (Fission-AI)	<i>Delta</i> (<code>Propose→Apply→Archive</code>); leve; sem Python	Descartado como 2ª ferramenta — a ideia do delta foi absorvida como raia leve
Kiro / Tessl / "specs as source of truth"	Variações de SDD assistido	Observados — mesmo princípio; sem adoção
Prompt ad-hoc	Sem spec	Rejeitado — produz código que compila e erra a intenção

4. Recomendação de utilização (1 humano + N agentes)

- **Uma ferramenta SDD só (Spec Kit).** A raia leve (modelo delta) mora dentro dela.
- **Três raias:** leve (bug/typo — o PR é o artefato, bug exige teste que reproduz), plena (feature/contrato — Spec Kit completo), infra (sempre plena + gates de reversibilidade).

- Regra de desempate: **na dúvida, é plena; infra/migração nunca são leves.**

5. Conexões

- [3] / [4] — a spec é o **contexto integrador** que sobrevive ao `/clear` e cruza toda fronteira de subagente (antídoto ao ótimo local).
- **DDD/hexagonal** — dá as fronteiras (bounded contexts) onde a spec corta o trabalho.
- [8] **DoR** — "spec pronta" é a Definition of Ready.

6. Insight da jornada e impacto no modelo

"Documentação atualizada" era o *sintoma*; a causa é a **inversão de dependência**. A crítica "bug dá pra simplificar" + a avaliação do OpenSpec **produziram mudança normativa real**: `modelo-operacional.md` **v1.1.0** (§3 raias; spec de infra com gates)

- **ADR 0005**. Diário: [2] .

7. Fontes

- GitHub Blog — *Spec-driven development with AI* (2025-09): <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>
- GitHub Spec Kit: <https://github.com/github/spec-kit>
- OpenSpec (Fission-AI): <https://github.com/Fission-AI/OpenSpec>
- Spec Kit vs OpenSpec: <https://intent-driven.dev/knowledge/spec-kit-vs-openspec/>

Capítulo 04 — Fluxo agentic e economia de contexto (elemento [3])

Como o trabalho roda: **explore → plan → code → commit**, subagentes, `/clear` e revisor em contexto fresco — quatro peças, uma restrição.

1. Pergunta central

O fluxo agentic tem quatro peças que *parecem* boas práticas soltas. **Qual restrição física única elas estão todas contornando?**

2. Fundamentação teórica

A janela de contexto do agente é **finita**, e a performance **degrada à medida que ela enche** — o agente "esquece" instruções antigas e erra mais. O inimigo não é a *falta* de contexto; é o **acúmulo** (becos sem saída, arquivos irrelevantes) que afoga o sinal.

As quatro peças são, todas, **economia de contexto**:

PEÇA	O QUE FAZ COM O CONTEXTO
explore→ plan →code	o <i>plan</i> comprime a exploração no essencial
subagentes	leem em janela separada; só volta o resumo — o principal fica magro
<code>/clear</code>	despeja ruído entre tarefas não relacionadas
revisor fresco	não contaminado pelos becos sem saída; vê só diff + critério

Limite (não exagerar): resetar demais esquece o básico; delegar demais leva o subagente ao **ótimo local** (otimiza a fatia, quebra o todo). Disciplina madura: **preserve o contexto que sustenta (load-bearing), descarte só o ruído**. O contexto integrador é a **spec/plan** ([2]).

3. Frameworks / abordagens avaliados

PADRÃO/PRÁTICA	O QUE OFERECE	VEREDITO
explore-plan-code-commit (Claude Code)	Separa pesquisa de execução → não resolve o problema errado	Adotado (raia plena)
Writer/Reviewer (contexto fresco)	Revisor sem viés de quem escreveu	Adotado — é a revisão independente da DoD
Subagentes	Investigação/verificação em janela isolada	Adotado — cortados por bounded context
Revisão adversarial do diff	Modelo fresco tenta refutar antes do "pronto"	Adotado — <code>/code-review</code> como gate

PADRÃO/PRÁTICA	O QUE OFERECE	VEREDITO
<code>/clear</code> / compactação	Reset de contexto entre tarefas	Adotado — higiene de contexto

4. Recomendação de utilização (1 humano + N agentes)

- **Plan comprime** a exploração antes de implementar (raia plena).
- **Subagentes por bounded context**, cada um recebendo a spec (contexto integrador).
- **Revisão independente em contexto fresco** é item obrigatório da DoD (todas as raia).
- Preservar o load-bearing (objetivo, invariantes, contratos); descartar o ruído.

5. Conexões

- [2] — mesma espinha: a spec é o que sobrevive ao reset e cruza fronteiras.
- [4] — o ótimo local abre a necessidade de orquestração (reduce + corte por costura).
- [1] — checkpoint/rewind é a reversibilidade aplicada ao contexto.

6. Insight da jornada e impacto no modelo

Do "falta de contexto" ao **acúmulo/foco**; das 4 peças a **uma ideia (economia de contexto)**; do "delegar" ao **ótimo local**. Confirmou decisões que já estavam no modelo (`/clear` entre tarefas, revisão em contexto fresco no §4) — o aprendiz codificou o *quê* antes de entender o *porquê*. Diário: [3].

7. Fontes

- Claude Code — *Best practices* (economia de contexto, subagentes, revisão adversarial): <https://code.claude.com/docs/en/best-practices>
- Anthropic — *Building effective agents*: <https://www.anthropic.com/engineering/building-effective-agents>

Capítulo 05 — Padrões de orquestração de agentes (elemento [4])

Como coordenar subagentes sem cair em ótimo local: **map-reduce cognitivo** ao longo do espectro **workflow ↔ agent**.

1. Pergunta central

Cinco subagentes devolvem cinco fatias, cada um com a spec em mãos. Elas não se encaixam sozinhas. **O que precisa acontecer depois — e quem, estruturalmente, faz isso?**

2. Fundamentação teórica

A peça que falta depois do *map* (workers em paralelo) é o **reduce cognitivo**: reconciliar as fatias (contratos, coerência com a spec). Não é `cat fatia1 fatia2` — é *julgamento*, e só pode ser feito por quem tem o **contexto global**: o **orquestrador**. Padrão: **orchestrator-workers**.

O corte é mais traiçoeiro que o reduce. O map-reduce clássico assume splits independentes; em código quase nunca são (porta e adaptador dividem um contrato). Se você corta nas **costuras erradas**, nenhum reduce salva. As costuras certas são os **bounded contexts** (DDD) — cortar ao longo das fronteiras arquiteturais é o que torna a paralelização segura.

Espectro workflow ↔ agent. *Workflow* = você fixa o caminho (previsível, foco, sinergia). *Agent* = o LLM decide o corte na hora (flexível, mas pode errar o corte — ótimo local — e varia). Regra de ouro: **use a menor autonomia que resolve** ("comece simples; adicione autonomia só quando o simples comprovadamente falha").

3. Frameworks / abordagens avaliados

PADRÃO	O QUE É	TIPO	ONDE VIVE NO MODELO
Prompt chaining	passos fixos em sequência	workflow	fluxo Spec Kit <code>specify->...->implement</code>
Routing	classifica a entrada → handler certo	workflow	as raias (§3)
Parallelization	fatias independentes / N tentativas (voting)	workflow	subagentes por bounded context
Orchestrator-workers	corte dinâmico + reduce	agent	papel Orquestrador (§4)
Evaluator-optimizer	gera → crítica → refina	workflow c/ loop	Writer/Reviewer + <code>/code-review</code>
Autonomous	LLM aberto + guarda-corpos	agent	evitado (YAGNI; exige guardrails pesados)

4. Recomendação de utilização (1 humano + N agentes)

- **Default é workflow** (chaining/routing/parallel) — barato, previsível, depurável.
- **Parallelize por bounded context**; o **Orquestrador (humano)** faz o reduce.
- Suba para orchestrator-workers/autonomous **só** quando o corte genuinamente não pode ser predefinido.
- **Melhor arquitetura → mais tempo no lado barato do espectro.** Boas fronteiras compram previsibilidade.

5. Conexões

- [3] — resolve o *ótimo local* deixado em aberto pelo fluxo agentic.
- [2] / **DDD** — as fronteiras onde se corta; a spec como norte de cada worker.
- [1] — evaluator-optimizer é o revisor independente institucionalizado.

6. Insight da jornada e impacto no modelo

Analogia do aprendiz: "**map-reduce, mas cognitivo**"; e o insight de que **fixar o corte** reduz escopo e ganha sinergia (workflow > agent quando o corte é conhecido). Sem impacto normativo direto; mapeia os padrões sobre papéis/raias já existentes. Diário: [4].

7. Fontes

- Anthropic — *Building effective agents* (workflow×agent; 6 padrões; menor autonomia): <https://www.anthropic.com/engineering/building-effective-agents>
- Claude Code — *Best practices* (subagentes, fan-out, Writer/Reviewer): <https://code.claude.com/docs/en/best-practices>

Capítulo 06 — Papéis e responsabilidades (elemento [5])

Preservar os contrapesos de um time — **decide ≠ executa ≠ verifica** — sem ter um time: papéis como modos de trabalho numa **matriz humano × agentes**.

1. Pergunta central

Num time, decidir, executar e verificar são **pessoas diferentes** — é justamente essa separação que cria o contrapeso. Você é **uma** pessoa + N agentes. **Como preservar os contrapesos sem colapsar os três numa só cabeça?**

2. Fundamentação teórica

Papéis são modos de trabalho, não cargos. Cada papel é exercido por humano, agente, ou os dois com gate. Num time minúsculo os papéis **acumulam-se** — o risco é perder os contrapesos ao concentrar tudo numa pessoa.

RACI dá o vocabulário: **R**esponsible (executa), **A**ccountable (responde/aprova), **C**onsulted (verifica/consultado), **I**nformed. Regra clássica: **exatamente um Accountable por tarefa** (um pescoço a apertar).

Delegabilidade em solo+IA:

- **R (executa)** → agente (dev/spec/plan/etc.).
- **C (verifica)** → agente **independente**, em contexto fresco ([3]) — nunca o mesmo que executou, nunca só o próprio humano (senão "eu decidi, eu confiro" mata a independência).
- **I (informado)** → logs / ExecutionTrace.
- **A (Accountable)** → **humano, sempre**. Um agente raciocina, executa e revisa, mas **não responde pelas consequências** — accountability não delega ([1]).

A armadilha do funil. Se o humano é A e confere à mão cada saída de agente, ele recria "fazer tudo sozinho" — o A vira gargalo. Resolução: **o humano é Accountable pela política, pelos gates e pelos critérios — não por cada instância.** Ele projeta os trilhos (allow/deny/ask + DoD verificável + critérios da spec); os agentes operam dentro deles; ele faz spot-check e responde pelos resultados. É a *política de delegação* do [1] aplicada a papéis — o que faz o modelo escalar.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Matriz RACI/RASCI	Quem executa/responde/consulta/informa por tarefa	Adotado — adaptado a humano × agentes
Regra "um Accountable"	Responsabilidade não se dilui	Adotado — o humano é o A único e fixo no risco

ABORDAGEM	O QUE OFERECE	VEREDITO
Papéis Scrum (PO/SM/Dev Team)	Papéis de time cross-funcional	Parcial — PO = Steward humano; Scrum Master e "dev team" são cerimônia de papel para solo (ver [6])
Time T-shaped / cross-funcional	Largura de skills + profundidade	Reinterpretado — o agente cobre a largura (muitas skills); o humano guarda a profundidade da decisão

4. Recomendação de utilização (1 humano + N agentes)

- Adotar a **matriz de papéis-modo** do `modelo-operacional.md` §4: Product Steward, Arquiteto/Tech Lead, Spec/Plan/UX/Dev/QA/Review/Security/Tech-Writer-agents, Orquestrador.
- **R/C/I → agentes; A → humano**, e o humano é Accountable **pela política/gates/critérios**, não por item.
- **C sempre independente do R** (contexto fresco).
- Indelegável ao agente: aprovar spec, plan, merge, autorizar deploy/migração.

5. Conexões

- [1] — accountability indelegável + política de delegação (o "de que" o humano é A).
- [3] — o revisor em contexto fresco **implementa** o C independente.
- [8] **DoR/DoD** — critérios verificáveis são o que **permite** delegar o C a um agente.
- [9] **Gates/risco** — a taxonomia de risco define onde o A **tem** que agir.

6. Insight da jornada e impacto no modelo

Insights do aprendiz: "**delega-se tudo menos o A**" (porque accountability responde pelas consequências) e "**o humano é Accountable pela política, gates e critérios — não por cada item**" (o que evita o funil e faz escalar). Sem impacto normativo novo — consolida e fundamenta a matriz RACI já presente no §4. Diário: [5] .

7. Fontes

- Anthropic — *Building effective agents* (revisão humana crucial em agentes de código): <https://www.anthropic.com/engineering/building-effective-agents>
- Matriz RACI — prática consolidada de atribuição de responsabilidades em gestão.
- `docs/governance/modelo-operacional.md` §4 (papéis + RACI); Capítulo 01 ([1]).

Capítulo 07 — Cerimônias e cadência (elemento [6])

Cadência **Shape Up + Kanban**, não Scrum: cerimônia é **função**, não reunião — e o WIP se mede em **atenção do humano**, não em capacidade de agentes.

1. Pergunta central

Quais cerimônias sobrevivem quando é **um humano + N agentes** — e, já que os agentes paralelizam, **qual é o limite real do WIP?**

2. Fundamentação teórica

Cerimônia é a função que entrega, não a reunião. Para decidir o que cortar, pergunte "essa **função** ainda existe no meu contexto?".

CERIMÔNIA	FUNÇÃO	EM SOLO+IA
Planning	comprometer escopo	vira shaping/spec ([2])
Daily	sincronizar/destravar entre pessoas	função some → cortada
Review	inspecionar incremento	vira checkpoint de ciclo
Refinement	preparar backlog	cortada (sem backlog — Shape Up)
Retro	melhorar o processo	amplificada (ver abaixo)

Retro amplificada. Num time humano, a melhoria da retro é um hábito mole que decai. Com agentes, ela vira **instrução versionada** (`CLAUDE.md/skill/constituição`): **durável e executável**, aplicada automaticamente na próxima. É a cerimônia de **maior ROI** — cada erro recorrente de agente vira regra.

Cadência = Shape Up + Kanban. *Apetite* (tempo fixo, escopo variável — o oposto de estimar prazo; casa com YAGNI e diffs pequenos), *cooldown* (dívida/manutenção/curadoria), sem *backlog* formal (ideias importantes voltam via re-shaping), fluxo contínuo com **WIP limitado**.

O limite real do WIP é a atenção do humano. Não é a capacidade dos agentes nem a dependência entre tasks (essa limita o paralelo **dentro** de uma feature). Mesmo com 5 features de dependência zero, cada uma funila pelos **gates de decisão/aprovação/verificação do humano** ([5]). Portanto:

- **paralelize DENTRO** de uma feature já decidida (subagentes por bounded context);
- **não paralelize ACROSS** features ambíguas (multiplicam os gates);
- para escalar, **barateie os gates** (DoD verificável [8], reversibilidade [1]) — não abra mais frentes.

3. Frameworks / abordagens avaliados

FRAMEWORK	PAPÉIS/EVENTOS	VEREDITO PARA SOLO+IA
Scrum	PO/SM/Dev; planning, daily, review, retro, refinement	Parcial — mantém-se só a <i>função</i> de planning (→ spec) e retro; SM e daily são cerimônia de papel
Kanban	sem papéis/eventos; visualizar fluxo + limitar WIP	Adotado — WIP limitado (pela atenção humana) e fluxo contínuo
Shape Up	shapers/builders; appetite, ciclo 6sem + cooldown, betting, sem backlog	Adotado (esqueleto) — appetite + cooldown + sem backlog
Scrumban	híbrido	Observado — não precisamos do overhead do Scrum que ele carrega

4. Recomendação de utilização (1 humano + N agentes)

- **Cerimônias mínimas:** shaping/spec (planning real), execução por appetite, checkpoint de ciclo, cooldown, **retro** (→ regra versionada).
- **Cortar** daily e sprint planning (função de sincronização entre pessoas inexistente).
- **WIP baixo**, medido em **fluxos de decisão que o humano segura com qualidade**; paralelismo vai para **dentro** da feature.
- **Appetite** (tempo fixo, escopo variável): quando acaba, corta-se escopo — não se estende prazo.

5. Conexões

- [5] — o gargalo é o humano Accountable; o WIP é atenção humana.
- [4] — paralelizar *dentro* da feature (subagentes por bounded context).
- [2] — appetite = escopo variável = a raia certa por mudança.
- [8] / [1] — baratear os gates (DoD verificável, reversibilidade) é como se escala.

6. Insight da jornada e impacto no modelo

Insights do aprendizado: **cerimônia = função** (não reunião); **retro é a que fica mais valiosa com agentes**; dependência de tasks limita o paralelo **dentro** da feature, mas **o humano (atenção/decisão) é o gargalo do WIP através** de features. Sem impacto normativo novo — fundamenta a cadência do `modelo-operacional.md` §5. Diário: [6].

7. Fontes

- Basecamp — *Shape Up* (appetite, ciclo, cooldown, betting): <https://basecamp.com/shapeup>
- DORA — *four keys* (lote pequeno; velocidade×estabilidade): <https://dora.dev/guides/dora-metrics-four-keys/>
- `docs/governance/modelo-operacional.md` §5; Capítulos 01 ([1]), 06 ([5]).

Capítulo 08 — Entregáveis e artefatos (elemento [7])

Um artefato custa **duas vezes** (escrever + manter). Só vale o custo se for um **input consumido com forcing function** — ou imutável.

1. Pergunta central

A lista clássica é enorme (PRD, spec, plan, ADR, RFC, design doc, journey, runbook, changelog, C4...). **O que faz um artefato valer seu custo duplo — e o que o distingue de "cerimônia de papel" que apodrece?**

2. Fundamentação teórica

Todo artefato custa para **escrever** e para **manter atualizado**. O que "dá contexto e direção" é necessário, mas **não** distingue — um design doc também dá direção e apodrece. O que distingue é **mecânico**:

Um artefato sobrevive quando é um **INPUT** que algo downstream consome, com uma **forcing function** que falha barulhento quando ele está velho — ou quando é **imutável** (registra um ponto no tempo e nunca muda, como o ADR).

- A **spec** vive porque o agente **gera código dela**: velha → código errado → quebra visível ([2]).
- O **teste** vive porque a **CI** o consome: velho → vermelho.
- O **ADR** vive por **imutabilidade**: nunca precisa de update.
- O **journey** é **vivo por gate**: a Constituição (Princípio VII) regenera as capturas do build real e revisa a heurística **no mesmo PR** — sem esse gate, apodrece. É o caso canônico de *living documentation* / *docs-as-code*: doc desconectado do fluxo não tem forcing function.

Segunda metade da regra: não duplique função já servida por um artefato vivo. Um **PRD** avulso duplica a função da **spec**; um **design doc/RFC** avulso duplica **plan + ADR**. Sem consumidor próprio, apodrecem → cerimônia de papel (YAGNI).

3. Frameworks / abordagens avaliados

ARTEFATO	CONSUMIDOR	FORCING FUNCTION	VEREDITO
Spec (<code>spec.md</code>)	agente (gera código)	código errado se velha	Essencial
Plan (<code>plan.md</code>)	dev-agent / Constitution Check	divergência da impl	Essencial
Tasks (<code>tasks.md</code>)	dev-agent	efêmero (descarta pós-uso)	Essencial (efêmero)
Código + testes	CI	vermelho	Essencial

ARTEFATO	CONSUMIDOR	FORCING FUNCTION	VEREDITO
ADR	decisões futuras / onboarding	imutável	Essencial
Journey doc	gate de PR (capturas + heurística)	PR falha se não regenerar (P. VII)	Essencial (por gate)
Runbook	incidente / deploy	uso real / drills (fraco)	Essencial (disciplina)
Changelog	release	PR de release	Essencial (se gated)
DoR/DoD	agente / Stop-hook	gate de merge	Essencial
PR	revisão / merge	—	Essencial (na raia leve, é o artefato)
PRD avulso	(duplica a spec)	nenhuma	YAGNI
Design doc / RFC avulso	(duplica plan + ADR)	nenhuma	YAGNI
C4 diagrams	humano (leitura)	nenhuma	Fase 2 / YAGNI
Painel de métricas DORA	—	—	YAGNI

4. Recomendação de utilização (1 humano + N agentes)

- **Manter** apenas artefatos com consumidor + forcing function (ou imutáveis).
- **Não criar** artefato cuja função já é servida por um vivo (PRD→spec; design doc→plan+ADR).
- **Journey exige o gate** do Princípio VII para não apodrecer (não é "doc solto").
- Formalizar o **changelog** com forcing function no PR de release (pendência do modelo).

5. Conexões

- **[2]** — a spec é o arquétipo do "input consumido que não apodrece".
- **[8] DoR/DoD** — são artefatos-checklist verificáveis (habilitam o gate barato).
- **[11]** — a rastreabilidade liga os artefatos (spec↔PR↔testes↔journey).
- **[12]** — ADR é o artefato imutável de governança.

6. Insight da jornada e impacto no modelo

O que faz um artefato valer não é o **conteúdo** ("dá contexto") e sim o **mecanismo: consumidor + forcing function**, ou **imutabilidade** (ADR). E **não duplicar função já servida** (PRD/design doc avulsos = YAGNI). Fundamenta o catálogo do `modelo-operacional.md` §6. Diário: **[7]**.

7. Fontes

- M. Nygard — *Documenting Architecture Decisions* (ADR):
<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- Simon Brown — *C4 model*: <https://c4model.com/>
- L. Mezzalana — *Documenting software architecture* (docs-as-code / living documentation):
<https://luamezzalana.medium.com/how-to-document-software-architecture-techniques-and-best-practices-2556b1915850>
- `docs/governance/modelo-operacional.md` §6; Constituição (Princípio VII, jornadas vivas).

Capítulo 09 — Definition of Ready / Done (elemento [8])

A DoD é o gate. Um gate barato é **verificável autonomamente** — mas verde ≠ certo.

1. Pergunta central

Baratear os gates é como se escapa do gargalo humano ([6]). **Que propriedade um critério de "done" precisa ter para um agente satisfazê-lo e um hook conferi-lo sem depender de você — e o que ele, mesmo verde, ainda não garante?**

2. Fundamentação teórica

A propriedade: verificável autonomamente. Um critério de "done" precisa produzir um **pass/fail objetivo** que o agente gera e um hook confere — "prove, não declare" ([11]) virado em máquina. *"Código limpo"* não é DoD (subjetivo); *"testes verdes + lint + revisão sem lacunas de correção"* é.

O trabalho é converter julgamento em check (harness design — ADR 0001):

- "código limpo" → lint + fitness functions + limite de complexidade;
- "boa UX" → tokens do design system + checklist heurístico;
- "seguro" → secret scanning + revisão de injeção/autorização.

DoR é a mesma ideia a montante: "pronto para começar" = a spec/plan é completa o bastante para o agente executar **sem adivinhar** (critérios testáveis, ambiguidades resolvidas, Constitution Check, apetite definido).

Necessária, não suficiente. Uma DoD verde garante a **peça local** — não garante:

- **(a) coerência global:** a jornada/todo ainda funciona e não regrediu (o **ótimo local** do [4] no nível da qualidade);
- **(b) a coisa certa:** verde não conserta requisito/spec errado. O modelo trata (a) puxando o global para dentro da DoD (journey, e2e, rastreabilidade [11]) + checkpoint humano; (b) fica com o **humano (o A do [5])**.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
DoR / DoD (Scrum/Kanban)	Portões de entrada/saída	Adotado — como checklists verificáveis (§7 do modelo)
Testing trophy (mais integração) vs test pyramid (mais unitário)	Onde investir teste	Trophy-leaning — integração por rota + contrato por porta pegam o "todo" melhor que só unitário
TDD / BDD	Teste antes; critério em linguagem de negócio	Adotado — bug exige teste que reproduz; caso de uso tem feliz + falha

ABORDAGEM	O QUE OFERECE	VEREDITO
Quality gates no CI + DevSecOps leve	Bloqueio automático	Adotado — fitness functions, lint/typecheck, secret scanning
Cobertura como meta numérica	% de linhas	Rejeitado — gamável; usamos "feliz + falha por caso de uso"

4. Recomendação de utilização (1 humano + N agentes)

- **DoR e DoD como checklists verificáveis** (`modelo-operacional.md` §7), fecháveis por Stop-hook/ `/goal` .
- **DoD reduzida na raia leve; bloco de reversibilidade** na raia infra.
- **Puxar o global para a DoD**: journey atualizada + e2e + rastreabilidade spec↔journey.
- **O humano guarda o irreduzível**: coerência global no checkpoint e "é a coisa certa".

5. Conexões

- `[1]` — "prove, não declare" é a raiz da DoD verificável.
- `[5]` — o humano é Accountable pelos **critérios**; a DoD é onde eles viram check.
- `[6]` — DoD verificável = gate barato = fuga do gargalo humano.
- `[4]` / `[11]` — verde local ≠ todo correto; rastreabilidade/journey puxam o global.

6. Insight da jornada e impacto no modelo

Insights do aprendizado: "**verificável autonomamente**" e "**a peça é local — ainda é preciso ver se atendeu o requisito da jornada / não comprometeu o conjunto maior**" (o ótimo local reaparecendo). Fundamenta a DoR/DoD do `modelo-operacional.md` §7. Diário: `[8]` .

7. Fontes

- Claude Code — *Best practices* (dê ao agente um check que ele rode; verificação como gate): <https://code.claude.com/docs/en/best-practices>
- DORA — *four keys* (testes/CI como capacidades): <https://dora.dev/guides/dora-metrics-four-keys/>
- Kent C. Dodds — *The Testing Trophy* (conceito; ênfase em integração).
- `docs/governance/modelo-operacional.md` §7; ADR 0001 (harness design).

Capítulo 10 — Gates humanos e classes de risco (elemento [9])

Um gate **uniforme** está sempre errado: pesado demais vira funil, leve demais deixa o irreversível escapar. O peso do gate escala com **irreversibilidade** x **impacto**.

1. Pergunta central

Você barateou os gates mecânicos ([8]) e guardou o humano para o global e o "é a coisa certa". Mas aprovar um tipo ≠ aprovar uma migração destrutiva. **Por que graduar o gate por risco em vez de um gate uniforme — e qual eixo decide o peso?**

2. Fundamentação teórica

Os dois fracassos do gate uniforme:

- **pesado em tudo** → você aprova até o trivial → vira o **funil** (o gargalo do [6]);
- **leve em tudo** → uma ação irreversível **escapa** sem aprovação → a migração do [1] .

O eixo: **irreversibilidade** x **impacto (blast radius)**. O gate deve ser **proporcional ao risco**: automatize o baixo, escale o alto. Taxonomia oficial (Constituição §IV; modelo §8):

CLASSE	EXEMPLO	AGENTE SOZINHO?	GATE
Leitura	explorar, buscar	✓	nenhum
Leitura sensível	PII/segredos	⚠ política + máscara	revisão
Criação reversível	feature em branch	✓	merge
Alteração	refactor amplo, contrato	✗	aprovação com resumo
Exclusão / externa	apagar dados, push, API externa	✗	confirmação forte
Financeira / irreversível	deploy, migração destrutiva	✗	dupla aprovação
Lote / cross-tenant / admin	migração em massa	✗ bloqueado	workflow humano formal

Dois amarres:

1. **Mesmo eixo das raias ([2])**. **ambiguidade** x **raio** x **irreversibilidade** decide quanto **processo** (spec); **irreversibilidade** x **impacto** decide quanto **gate**. Mesma física: *processo* para construir, *gate* para agir.
2. **Reversibilidade rebaixa a classe**. Tornar a ação reversível (backup/staging/ soft-delete — [1]) move-a escada de risco **abaixo** → gate mais leve → menos funil.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Política <code>allow/deny/ask</code> (OpenAI Agents approvals; OPA)	Decisão declarativa por classe	Adotado — a máquina de estados de ação (Const. §IV)
Taxonomia de classes de risco (Const. §IV)	Grada o gate	Adotado — base do <code>[9]</code>
Human approval for high-stakes (Anthropic)	Humano no ponto consequente	Adotado — gates indelegáveis
Least privilege / RBAC-ABAC	Autorização fora do LLM	Adotado — política decide, não o modelo
Gate uniforme (um approve pra tudo)	Simplicidade aparente	Rejeitado — funil ou catástrofe

4. Recomendação de utilização (1 humano + N agentes)

- **Gate proporcional** pela taxonomia do modelo §8; automatizar baixo risco, escalar alto, **bloquear** lote/cross-tenant/admin.
- **Indelegáveis** (sempre humano): aprovar spec, plan, merge, autorizar deploy/migração.
- **Autorização fora do LLM** (RBAC/ABAC/política) — o modelo nunca decide permissão.
- **Investir em reversibilidade** para rebaixar classes e reduzir gates pesados.

5. Conexões

- `[1]` — reversibilidade é o eixo e a alavanca (rebaixa a classe).
- `[2]` — mesma física das raia, aplicada a ações.
- `[5]` — o A humano age exatamente nos gates altos; delega os baixos.
- `[6]` / `[8]` — gate barato/automático no baixo risco evita o funil.

6. Insight da jornada e impacto no modelo

Insights do aprendiz: o eixo é **irreversibilidade x impacto**; um gate uniforme é funil ou catástrofe; logo **gate proporcional**. Fundamenta o mapa de gates do `modelo-operacional.md` §8 e a taxonomia da Constituição §IV. Diário: `[9]`.

7. Fontes

- Anthropic — *Building effective agents* (revisão humana para alto risco): <https://www.anthropic.com/engineering/building-effective-agents>
- OWASP — *LLM01 Prompt Injection* (dados também são hostis; política fora do LLM): <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- Open Policy Agent: <https://www.openpolicyagent.org/docs>
- Constituição (Princípio IV, classes de risco); `docs/governance/modelo-operacional.md` §8.

Capítulo 11 — Rastreabilidade

spec ↔ PR ↔ testes ↔ journey (elemento [11])

Não é burocracia: é a **memória de longo prazo** do projeto — o porquê e o quê, sem re-derivar — e **emerge** do workflow, sem ferramenta pesada.

1. Pergunta central

Daqui a 3 meses um teste quebra e o **agente tem memória zero**. **O que a rastreabilidade te dá nesse momento — e como tê-la sem uma matriz de compliance que ninguém lê?**

2. Fundamentação teórica

Rastreabilidade devolve **o quê** (o que foi construído, o que o verifica) e **o porquê** (a decisão e o racional) **sem re-derivar**.

Por que é load-bearing em solo+IA: o agente **reseta** o contexto ([3]). Os artefatos ligados são a **única memória durável** — a rastreabilidade é o que reconstrói intenção + verificação depois do reset.

Dois sentidos:

- **para frente:** spec → foi construída? está verificada? (cobertura de intenção);
- **para trás:** sintoma → qual código → qual spec → por quê — que é **recuperação rápida** (a métrica de estabilidade do [10]).

Emergente, não um sistema. Você não constrói uma ferramenta de rastreabilidade; o elo **emerge** de cada artefato referenciar o próximo — spec **NNN** → PR cita a spec → teste nomeia o requisito → journey no mesmo PR → ADR para a decisão. A **DoD ([8])** **força o elo**. É subproduto do workflow, não atividade separada.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Matriz de rastreabilidade de requisitos	Rastreio formal req↔teste	Rejeitado — ferramenta pesada; matriz que ninguém mantém
Linking nativo issue↔commit↔PR (GitHub)	Elo barato no fluxo	Adotado — spec NNN e PR se referenciam
ADR ligado a componentes/C4	Decisão → sistema	Parcial — ADR sim; C4 é Fase 2
docs-as-code (cross-refs)	Links versionados	Adotado — journey/ADR/spec se citam

4. Recomendação de utilização (1 humano + N agentes)

- **Elo obrigatório na DoD:** spec **NNN** ↔ PR ↔ testes ↔ journey explícito.

- **Convenção de referência** (a spec numera; o PR cita; o teste nomeia o requisito; o ADR registra a decisão) — sem matriz, sem ferramenta.
- Otimizar o **sentido para trás** (sintoma → causa) — é o que acelera a recuperação.

5. Conexões

- [3] — os artefatos ligados são a memória que sobrevive ao reset de contexto.
- [8] — a DoD carrega e força o elo de rastreabilidade.
- [10] — o sentido "para trás" é a recuperação rápida (estabilidade).
- [7] / [12] — artefatos consumidos + ADR imutável = a trilha de decisões.

6. Insight da jornada e impacto no modelo

Insight do aprendiz: a rastreabilidade dá "**o porquê e o quê**" sem re-derivar; em solo+IA os artefatos ligados **são a memória** do projeto. Fundamenta o §8 (rastreabilidade) e a DoD (§7) do `modelo-operacional.md`. Diário: [11].

7. Fontes

- DORA — *four keys* (recuperação = trilha do sintoma à causa): <https://dora.dev/guides/dora-metrics-four-keys/>
- L. Mezzalana — *Documenting software architecture* (docs-as-code, cross-refs): <https://lucamezzalana.medium.com/how-to-document-software-architecture-techniques-and-best-practices-2556b1915850>
- `docs/governance/modelo-operacional.md` §8–§9.

Capítulo 12 — Governança leve (elemento [12])

Como o próprio sistema evolui e se mantém coerente — **aprendendo sem inchar**. A camada meta que fecha a jornada.

1. Pergunta central

Você tem um sistema de regras que **cresce** (constituição, ADRs, modelo versionado). **Qual é o modo de falha de um sistema de regras que cresce — e que força o combate?**

2. Fundamentação teórica

Modo de falha: bloat / ossificação. Regras acumulam, ninguém poda, e o conjunto vira o **processo pesado que a gente cortou** lá no começo.

A força contrária: YAGNI + poda + flexibilidade. A governança precisa de duas forças opostas equilibradas:

- **aprender** (adicionar regra) — via retro → ADR → nova versão;
- **ficar lean** (remover/não adicionar) — via YAGNI e poda do que não paga.

A Constituição já carrega essa poda: *"complexidade além do necessário DEVE ser justificada por escrito ou removida (YAGNI)."*

A arquitetura que dá flexibilidade: camadas com velocidades diferentes.

- **Núcleo firme** — a **Constituição** (princípios) muda devagar, por emenda versionada com Sync Impact Report;
- **Periferia evoluível** — **modelo operacional + handbook**, com **versão própria**, mudam rápido (por isso ancorados como *extensão subordinada* — Constituição v1.4.0);
- **Memória append-only** — **ADRs** imutáveis ([7]): o log de decisões que não apodrece;
- **Loop de aprendizado** — retro → regra versionada ([6]).

Governança aplica os próprios princípios a si mesma: é versionada, emendada sob gate humano ([9]), registrada em ADR ([7]) e podada por YAGNI. E os ADRs, sendo imutáveis mas **substituíveis** (um novo ADR pode superar outro), dão à decisão a mesma **reversibilidade** do [1] : nada fica preso, tudo é rastreável.

3. Frameworks / abordagens avaliados

ABORDAGEM	O QUE OFERECE	VEREDITO
Constituição versionada (spec-kit)	Fonte de verdade + emenda disciplinada	Adotado — núcleo firme
ADR (Nygard)	Log imutável de decisões + racional	Adotado — memória de governança
Working agreements / handbook	Convenções evoluíveis	Adotado — periferia rápida (este handbook)

ABORDAGEM	O QUE OFERECE	VEREDITO
RFC process pesado	Revisão formal ampla	Rejeitado — ADR cobre; YAGNI
Governança "adicionar sempre"	Mais regras = mais controle	Rejeitado — leva a bloat/ossificação

4. Recomendação de utilização (1 humano + N agentes)

- **Constituição firme em princípios**; modelo/handbook **evoluíveis e subordinados**.
- **ADR para toda decisão** que muda o sistema (imutável; superável por outro ADR).
- **Emenda via retro** (erro recorrente → regra); **poda via YAGNI** (regra que não paga sai).
- **Revisão obrigatória ao fim de cada fase** do projeto.

5. Conexões

- É a **meta-camada** de todos os elementos.
- [6] — a retro é o motor de emenda; [7] — ADR é a memória imutável; [9] — emenda é human-gated; [1] — ADR superável = reversibilidade da decisão.

6. Insight da jornada e impacto no modelo

Insights do aprendizado: **bloat/ossificação** é o risco; **YAGNI + podar + flexibilidade** é a força; governança precisa ser flexível para evoluir. **Meta**: esta própria jornada de aprendizado foi o [12] em ação — crítica → refino do modelo → ADR 0005 → versão v1.1.0 → ancoragem na Constituição v1.4.0. Diário: [12] .

7. Fontes

- M. Nygard — *Documenting Architecture Decisions*: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- GitHub Spec Kit (constituição versionada): <https://github.com/github/spec-kit>
- [.specify/memory/constitution.md](#) (Governance); [docs/adr/](#); [docs/governance/modelo-operacional.md](#) §10-§11.